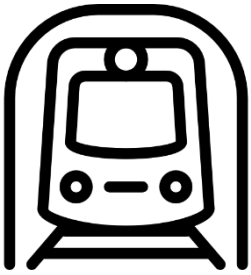
Thème : La Ville

Aux Lazaristes
Lyon - La Salle

Métroptimisation :
recherche du plus court chemin sur le réseau de métro Lyonnais



REYNIER Damien – MONDONGOU Gabin

PT2

2022 - 2023

Table des matières

Table des matières 3
I) Introduction 5
II) Acquisition des données 6
0) Organisation du travail 6
1) Utilisation de l'API (Application Programming Interface) 6
2) Création de la matrice d'adjacence 7
3) Présentation de notre graphe 8
III) Traitement des données 11
1) Algorithmes et codage 11
1.1 L'algorithme de Dijkstra 11
1.2 L'algorithme A* 13
2) Exécution et tests des programmes 14
2.1 Hypothèses de travail 14
2.2 Programme de tests 15
2.3 Résultats des tests 16
3) Analyse des résultats, interprétation 16
IV) Interface utilisateur 17
1) Page d'accueil 17
2) Page des résultats 19
V) Conclusion 19
Références bibliographiques 22
Annexe 1 : Diagramme d'exigences 23
Annexe 2 : Fichier Vérification.py 24
Annexe 3 : Fichier Algo_PCC.py 25
Annexe 4 : Interface utilisateur 28
Résumé 29
Abstract 29

I) Introduction

Avec le thème de la ville se pose la question de la mobilité. A l'ère du smartphone dans une société où tout doit aller vite et avec l'importance grandissante de l'utilisation des transports en commun pour des questions écologiques, optimiser l'utilisation du réseau, pour un trajet donné, répond aux enjeux de déplacements urbains.

Une application permettant d'optimiser les trajets en transport en commun, proposant le plus court chemin entre deux points de la métropole de Lyon serait un outil appréciable pour de nombreux utilisateurs.

La recherche du plus court chemin est l'un des problèmes les plus courants de l'algorithmique de la théorie des graphes et de l'optimisation combinatoire[1]. Elle consiste à trouver un chemin d'un sommet à un autre afin de minimiser la somme des poids des arêtes traversées.

Leonhard Euler, mathématicien, physicien et ingénieur suisse qui a vécu au 18e siècle, a introduit la théorie des graphes par ses travaux, en étudiant différents problèmes tels que les Ponts de Königsberg, le tour du chevalier sur un échiquier, ou encore le problème de la coloration des cartes. La théorie des graphes s'est ensuite beaucoup développée au 20e siècle grâce aux travaux de N. Biggs, C. Berge, W.T. Tutte, et d'autres [2].

Les graphes sont des structures mathématiques définies par un ensemble de sommets et d'arêtes qui relient ces sommets. Ils permettent de représenter et de modéliser les relations de connexion, de communication ou d'interaction entre différents éléments d'un ensemble. Ils sont utilisés pour modéliser de nombreux problèmes dans plusieurs disciplines telles que les mathématiques, l'informatique, la recherche opérationnelle et la chimie [2,3].

Bien que la recherche du plus court chemin puisse être facilement résolu avec des algorithmes polynomiaux [4], l'ajout de contraintes supplémentaires le rend NP-difficile.

Il existe deux grandes familles d'algorithmes pour résoudre ce problème : les algorithmes polynomiaux et les algorithmes d'énumération [5]. Des méthodes heuristiques d'approximation [5,6] ont également été proposées pour résoudre des problèmes impossibles à résoudre avec des algorithmes exacts.

Différents types d'algorithme peuvent être codés pour déterminer le plus court chemin. Ces algorithmes ont des complexités différentes et peuvent être plus ou moins rapides en temps d'exécution.

Il semble donc pertinent d'étudier dans quelle mesure la complexité d'un algorithme permet d'améliorer la recherche du plus court chemin, et en quoi le type de stockage des données influence le temps d'exécution du programme.

Dans un premier temps, nous présenterons les données d'entrée nécessaires à la réalisation de notre projet et caractériserons le graphe utilisé. Ensuite, nous exposerons et expliquerons les algorithmes codés pour notre travail, puis nous

réaliserons différents tests et analyserons les résultats obtenus. Enfin, nous expliquerons la mise en place et l'interface de notre application.

II) Acquisition des données

0) Organisation du travail

En premier lieu, nous avons réalisé un diagramme d'exigences afin d'encadrer les différents objectifs de notre projet (cf. Annexe 1).

Pour ce projet nous avons mis en place un répertoire sur le site **Github.com** qui nous a permis de mettre en commun notre travail afin de pouvoir travailler sur les mêmes documents en même temps tout en gardant des archives de versions antérieures de notre code. Ce répertoire est accessible à l'adresse suivante : **https://github.com/Gaboto74/TIPE**

Nous y ferons référence tout au long du rapport.

De manière à garder un code lisible et compréhensible, nous avons créé nos propres modules afin de séparer le code en plusieurs fichiers qui contiennent chacun quelques fonctions. Ainsi la méthode import nous permet d'importer les différentes fonctions crées dans le fichier principale Main.py.

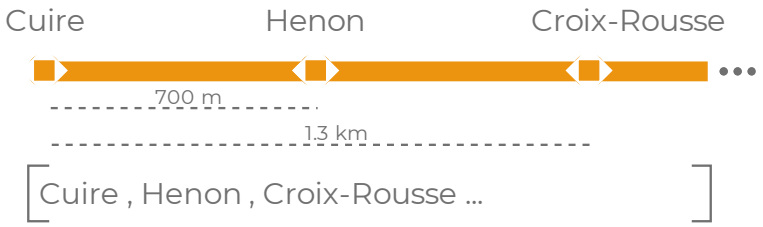
1) Utilisation de l'API (Application Programming Interface)

La Métropole de Lyon publie le site en ligne **data.grandlyon.com** permettant de récupérer différentes informations sur la ville tel que le cadastre, les chemins de randonnées et d'autres données accessibles à tous. Nous utilisons dans le cadre de notre projet les informations issues du réseau de transport en commun lyonnais. Deux jeux de données seront utiles :

- Les points d'arrêt du réseau Transports en Commun Lyonnais
- Les lignes de métro et funiculaire du réseau Transports en Commun Lyonnais

Le premier contient une liste de dictionnaire de tous les points d'arrêts, y compris ceux des bus qui ne seront pas utilisés pour notre projet, et le deuxième contient les lignes de métro et de funiculaire sous forme de tracés géométriques se basant sur un système de coordonnées précis. La première étape consistait donc à ordonner ces données de manière à obtenir pour chaque ligne, la liste des stations parcourues, dans le bon ordre. Nous avons donc créé un fichier Recuperation_données.py qui, aura pour objectif de récupérer les données à partir de l'API et d'en ressortir celles utiles. Le module requests nous permet de récupérer les données directement à partir d'un lien http, il faudra donc que l'application soit connectée à internet pour fonctionner. Après avoir supprimé les stations de bus (cf. ligne 48) et créé le dictionnaire lignes_infos ayant pour clés les lettres des lignes et

pour valeurs leurs informations associées dont la liste de leurs stations, il nous fallait organiser ces dernières dans le bon ordre. Pour cela nous avons débuter par mettre la première station au début de la liste car celle-ci était donnée par le deuxième jeu de données. Ensuite, la distance entre cette première station et toute les autres a été calculée afin de connaitre la station la plus proche, que nous placerons donc en deuxième position. En répétant ce processus, nous pouvions donc obtenir les listes des stations dans le bon ordre. (cf. ligne 80 et 114)



Ainsi, toutes les données nécessaires à notre projet étaient à notre disposition. La fonction retourne un tuple de trois objets :

- stations_infos : dictionnaire qui relie le nom de la station à ses informations correspondantes
- lignes_infos : dictionnaire qui relie la lettre de la ligne à ses informations correspondantes (dont ses stations)
- codes_lignes : dictionnaire qui relie la lettre de la ligne a son code numérique utilisé dans les jeux de données.

A partir de ces données, nous pouvions donc créer une matrice d'adjacence.

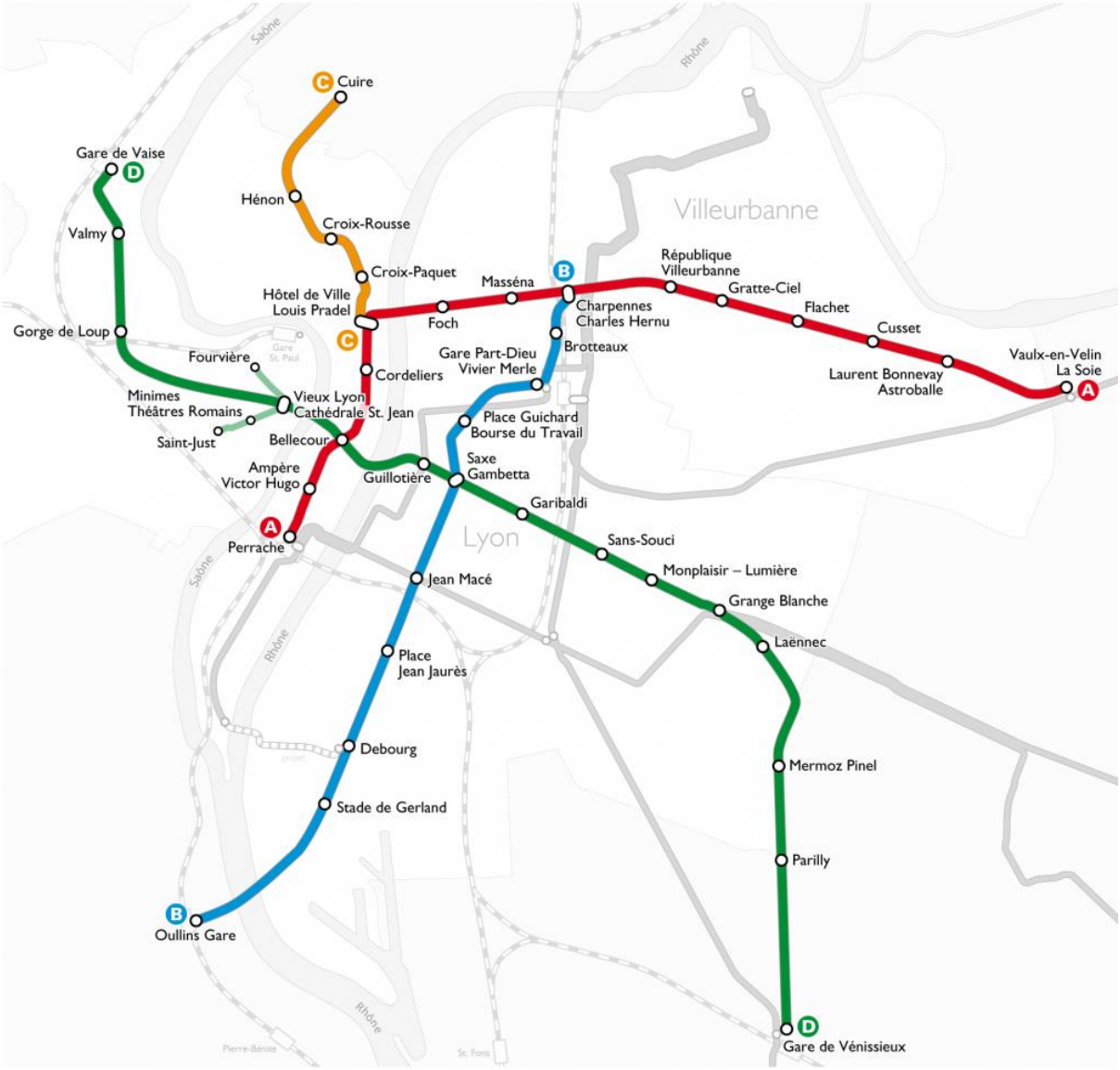
2) Création de la matrice d'adjacence

Les métros, pouvant se déplacer dans les deux sens, la matrice résultante sera symétrique ce qui signifie que le graphe correspondant ne sera pas orienté. Pour accomplir cela, nous avons créé une nouvelle fonction creation_matrice() dans un fichier distinct appelé Matrice_adjacence.py. Cette fonction prend deux arguments : lignes_infos et stations_infos. En utilisant la bibliothèque Numpy, nous avons créé une matrice carrée de zéros de taille égale au nombre de stations. Ensuite, nous avons trié les noms des stations par ordre alphabétique et les avons placés dans une liste, de telle sorte que l'indice d'une station dans cette liste correspond à son indice dans la matrice. Il est donc nécessaire de retourner cette liste en plus de la matrice. Puis nous remplissons chaque case de la matrice par la distance à vol d'oiseau qui sépare les stations entre elles. Nous faisons donc ici une approximation : le chemin entre deux stations est considéré comme une ligne droite. Les coordonnées des stations sont dans un système de coordonné spécial : ESPG. Il faudra donc par la suite modifier les distances calculées pour les convertir en mètres qui pourront être retrouvées à l'aide d'un coefficient multiplicateur.

3) Présentation de notre graphe

En amont du travail de recherche de plus court chemin, nous avons réalisé quelques tests sur notre graphe afin de mieux comprendre sa structure et d'en savoir plus sur sa nature et ses caractéristiques. Cela a été réalisé dans le but d'être plus apte à choisir la manière de coder nos futurs algorithmes.

Nous avons fait le choix de travailler seulement sur le réseau des métros. Le plan des métro, qui compose notre graphe et que nous avons codé par une matrice d'adjacence dans notre programme est présenté ci-dessous.



Source : [GES Lyon - Plan du métro, découvrir le métro de Lyon]

Il est nécessaire de bien connaître le type de graphe que l'on souhaite étudier. C'est pourquoi nous avons vérifié différentes caractéristiques des graphes dans le but de définir plus précisément notre graphe.

Connexe :

Un graphe est dit connexe si, pour tout couple de sommet du graphe, il existe un chemin reliant ces deux sommets. Autrement dit, il n'y a pas de sommets isolés dans le graphe et tous les sommets sont reliés d'une manière ou d'une autre. Un graphe non-connexe est forcément composé de plusieurs composantes connexes, c'est-à-dire de sous-graphes connexes qui ne sont pas reliés entre eux.

Pour tester cet aspect de notre graphe, nous avons utilisé l'algorithme connexite() (cf. Annexe 2)utilisant parcours_profondeur() (cf. Annexe 2) afin de voir si tous les sommets sont atteignables depuis le sommet 0. Etant donné que notre graphe est non orienté, si chaque sommet est atteignable depuis 0 alors tous les sommets atteignables entre eux.

Le résultat montre que notre graphe est bien connexe, il existera donc, à chaque fois, un plus court chemin entre deux stations de métro différentes.

Complet :

Un graphe complet est un graphe non orienté dans lequel chaque paire de sommets distincts est reliée par une arête. Autrement dit, dans un graphe complet de n sommets, chaque sommet est relié à tous les autres n-1 sommets par un arête. Un graphe complet est également appelé graphe totalement connecté. Le nombre d'arêtes dans un graphe complet est n(n-1)/2.

Etant donné que le nombre d'arêtes n'est clairement pas de n(n-1)/2, notre graphe ne peut pas être complet

Acyclique :

Un graphe acyclique ne contient pas de cycle c'est-à-dire qu'il est impossible de parcourir une suite d'arêtes pour revenir au même sommet. En d'autres termes, il n'y a pas de boucles dans le graphe.

Notre graphe n'est pas acyclique. En effet, un cyclique se trouve au milieu de notre graphe et relis Hôtel de ville, Charpennes, Saxe Gambetta et Bellecour ensemble.

Hamiltonien et Eulérien :

En opposition, nous retrouvons les graphes Eulérien et Hamiltonien.

Un graphe est dit Eulérien s'il contient un cycle qui passe par chaque arête exactement une fois. Ce cycle est appelé cycle Eulérien. Un graphe Hamiltonien est un graphe qui contient un cycle Hamiltonien c'est-à-dire un cycle qui passe exactement une seul fois par chaque sommet Pour le problème connu du voyageur de commerce [7,8], le graphe utilisé doit impérativement être Hamiltonien.

La condition nécessaire pour qu'un graphe soit Hamiltonien est que chaque paire de sommets non adjacents ait une somme de degrés supérieure ou égale au nombre total de sommets divisé par 2. Nous avons donc codé un algorithme, condition_Dirac() (cf. Annexe 2), permettant de vérifier cette condition. Le résultat obtenu montre que le graphe ne contient pas de cycle Hamiltonien ce qui nous

restreint très légèrement sur le choix des algorithmes de recherche de plus court chemin à tester.

Régulier :

Un graphe est dit régulier si tous ses sommets ont le même degré, c'est-à-dire le même nombre d'arêtes incidentes. On parle de graphe k-régulier si chaque sommet est relié à k autres sommets.

La plupart des stations n'ont que deux voisins mais ce n'est pas le cas de tous. En effet, Charpennes en a cinq. C'est pourquoi, le graphe n'est pas régulier.

Multigraphe/Simple :

Un multigraphe est un graphe doté d'une ou plusieurs arêtes multiples ou de boucles. En d'autres termes, il permet la présence de plusieurs arêtes entre deux sommets.

Un graphe simple est l'opposé d'un multigraphe. Autrement dit, c'est un graphe ne possédant ni boucles, ni arêtes multiples.

Notre graphe ne contient ni arêtes multiples, ni boucles ce qui signifient que c'est un graphe que l'on peut qualifier de simple.

Dense/ Creux :

Un graphe dense est un graphe dans lequel le nombre d'arêtes est proche du nombre maximal d'arêtes possible. Un graphe est généralement considéré comme dense si le nombre d'arêtes est de l'ordre de n^2 avec n le nombre de sommets. A l'inverse, un graphe creux est un graphe ayant relativement peu d'arêtes. Plus précisément, un graphe est considéré comme creux s'il a un nombre d'arêtes de l'ordre de grandeur de son nombre de sommets.

A l'aide d'un petit algorithme densite() (cf. Annexe 2) permettant de calculer le nombre d'arêtes présentent dans notre graphe, nous avons pu conclure sur la densité de notre graphe. Celui-ci est donc un graphe plutôt creux puisqu'il contient seulement une petite quarantaine d'arêtes.

Centralité :

La centralité d'un graphe mesure l'importance relative de chaque sommet dans le réseau. Il existe plusieurs types de centralités, en voici deux qui pourraient nous intéresser :

- Centralité de degré : mesure le nombre de liens directs qu'un nœud a avec les autres nœuds. Un nœud ayant un degré élevé est considéré comme important car il peut facilement interagir avec plusieurs autres nœuds.
- Centralité de proximité : mesure la distance moyenne entre un nœuds donné et tous les autres nœuds du réseau. Un nœud ayant une centralité de proximité élevée est important pour l'interaction avec d'autres sommets.

Dans le but de trouver les nœuds principaux et importants de notre graphe, nous avons tester la centralité en degré. Pour ce faire, nous avons coder un algorithme

centralite() (cf. Annexe 2) qui nous retourne les sommets avec plus de deux voisins et nous a permis de connaitre les cinq nœuds principaux : Hôtel de ville, Charpennes, Saxe Gambetta, Bellecour et Vieux Lyon.

Nous pouvons donc conclure que notre graphe est connexe, ce qui est indispensable pour la recherche de chemin. Il est également simple, ce qui implique que le chemin trouvé par l'algorithme sera l'unique plus court chemin entre un point A et un point B. Ensuite il est creux, ce qui signifie que le nombre de chemin à tester sera raisonnable, ce qui implique que la recherche de chemin devrait être assez rapide. Enfin, nous avons identifié cinq nœuds principaux.

III) Traitement des données

Nous avons choisi de coder deux algorithmes différents, un algorithme de Dijkstra de type polynomiale et un algorithme A* qui utilise une évaluation heuristique, et de les tester afin de déterminer lequel des deux serait le plus efficace pour répondre à notre problématique.

1) Algorithmes et codage

1.1 L'algorithme de Dijkstra

L'algorithme de Dijkstra est un algorithme de recherche de chemin le plus court dans un graphe pondéré inventé par l'informaticien néerlandais Edsger W. Dijkstra en 1956. [9]

Le fonctionnement de l'algorithme de Dijkstra peut être décrit en plusieurs étapes :

a. Initialisation : L'algorithme de Dijkstra commence par initialiser la distance de tous les sommets du graphe à l'infini, sauf pour le sommet de départ qui est initialisé à 0. On marque également tous les sommets comme « non visités » pour l'instant (cf. ligne 42 à 47, Algo_PCC.py).

b. Sélection du sommet le plus proche : À chaque étape, l'algorithme de Dijkstra sélectionne le sommet non visité le plus proche du sommet de départ. Le sommet de départ est considéré comme le premier sommet sélectionné (cf. ligne 51 et 55).

c. Mise à jour des distances : Une fois que le sommet le plus proche est sélectionné, l'algorithme met à jour la distance de ses voisins en additionnant la distance du sommet sélectionné et le poids de l'arête qui les relie. Si la distance mise à jour est plus petite que la distance actuelle de ce voisin, alors la distance actuelle est remplacée par cette nouvelle distance (cf. ligne 56 à 58).

d. Marquage du sommet visité : Une fois que tous les voisins du sommet sélectionné ont été mis à jour, le sommet sélectionné est marqué comme « visitée » (cf. ligne 52).

e. Répétition : Les étapes b. à d. sont répétées jusqu'à ce que tous les sommets du graphe soient visités ou que le sommet d'arrivée soit atteint.

f. Retour du chemin : Une fois que l'algorithme de Dijkstra a atteint le sommet d'arrivée, il reconstruit le chemin le plus court en partant du sommet d'arrivée et en remontant les distances mises à jour jusqu'au sommet de départ (cf. ligne 64 à 71).

L'algorithme de Dijkstra est un algorithme optimal qui garantit de trouver le chemin le plus court entre le sommet de départ et le sommet d'arrivée. C'est le premier que nous avons implémenté dans notre application.

Il possède, en général, une complexité en O((a+n)log(n)) avec a le nombre d'arrêtes et n le nombre de sommets. C'est donc un algorithme efficace pour un graphe de taille moyenne comme le nôtre. En effet, il est important de noter que cet algorithme peut vite devenir inefficace pour des graphes de très grande taille car il nécessite de stocker et de mettre à jour les distances de tous les sommets du graphe à chaque étape.

Dans un premier temps, nous avons implémenté la fonction dijkstra_mat() (cf. Annexe 3) avec une complexité en $O(n^2)$. Cette complexité est due au stockage des nœuds et des arêtes dans une matrice d'adjacence, ce qui ne permet pas une efficacité maximale de l'algorithme de Dijkstra. Cependant, il est possible d'améliorer cette complexité en utilisant une structure de données plus efficace pour stocker les nœuds, comme la mise en place de listes de priorité.

Une deuxième version de l'algorithme de Dijkstra, appelée dijkstra_dic() (cf. Annexe 3), a donc été implémentée. Elle utilise une liste de priorité de la bibliothèque heapq pour stocker les nœuds et les arêtes du graphe. Cette nouvelle implémentation a une complexité en O((a+n)log(n)), où a est le nombre d'arêtes et n le nombre de sommets.

La bibliothèque heapq utilisée, est une bibliothèque Python qui fournit des fonctions pour travailler avec les files ou tas de priorité en utilisant la structure de données de tas binaire. C'est une bibliothèque très efficace en termes de temps d'exécution et d'utilisation de la mémoire, grâce à la structure de données de tas binaire qui maintient la propriété de chaque nœud à une clé inférieure ou égale à la clé de chacun de ses enfants. Cela garantit donc que l'élément de la plus petite valeur est toujours en tête de file de priorité et peut être extrait en temps constant.

Dans le programme dijkstra_dic(), deux commandes de heapq sont utilisées :

- Heapq.heappop(heap) : extrait et retourne le plus petit élément (appelé la racine) de la heap. Dans notre cas, il permet d'extraire le sommet avec la distance la plus courte (cf. ligne 97).
- Heapq.heappush(heap, (distance_temp, voisins)) : ajoute un élément au heap en maintenant la propriété que la racine est toujours le plus petit

élément. Dans notre cas, il permet d'ajouter un nouveau sommet et sa distance à la heap (cf. ligne 115).

1.2 L'algorithme A*

L'algorithme A* lu « A étoile » ou « A star » en anglais, proposé par Peter Hart, Nils Nilsson et Bertram Raphael en 1968,est un algorithme de recherche de plus court chemin qui utilise une fonction heuristique pour améliorer l'efficacité de la recherche [10].

En informatique, l'heuristique est une méthode qui permet de trouver des solutions approchées et souvent satisfaisantes à des problèmes complexes. Contrairement à une méthode algorithmique qui garantit une solution exacte, une heuristique est une méthode qui peut conduire à une solution qui n'est pas optimale, mais qui est néanmoins suffisante pour la plupart des cas pratiques et surtout de manières bien plus rapide qu'une recherche exacte.

La fonction heuristique utilisée par l'algorithme A* est une estimation de la distance restante entre chaque nœud et le nœuds d'arrivée. Dans notre cas, puisqu'il s'agit d'une carte, nous avons utilisé la distance à vol d'oiseau entre chaque nœud. Cette fonction heuristique va permettre de guider la recherche de chemin du nœud d'arrivée plus rapidement en explorant les nœuds les plus prometteurs en premier.

Il est important de noter que l'efficacité de l'algorithme A* dépend grandement de la qualité de la fonction heuristique. En effet, si la fonction heuristique est trop optimiste, cela peut conduire à une exploration inutile de chemins qui n'aboutiront pas à la solution. Dans l'autre sens, si la fonction est trop pessimiste, cela peut ralentir la recherche en explorant des nœuds inutilement.

Le fonctionnement de l'algorithme A* peut être décrit en plusieurs étapes :

a. Initialisation : L'algorithme A* commence par initialiser la distance de tous les sommets du graphe à l'infini, sauf pour le sommet de départ qui est initialisé à 0. On marque également tous les sommets comme « non visités » pour l'instant. En plus de cela, A* utilise une estimation de la distance restante jusqu'à la destination appelée heuristique (cf. ligne 122 à 126, Algo_PCC.py).

b. Sélection du sommet le plus prometteur : À chaque étape, l'algorithme A* sélectionne le sommet non visité qui a la plus petite valeur de coût total, qui est la somme de la distance du sommet de départ et de l'estimation de la distance restante jusqu'à la destination (l'heuristique). Le sommet de départ est considéré comme le premier sommet sélectionné (cf. ligne 130 à 134).

c. Mise à jour des distances : Une fois que le sommet le plus prometteur est sélectionné, l'algorithme met à jour la distance de ses voisins en additionnant la distance du sommet sélectionné et le poids de l'arête qui les relie. Si la distance mise à jour est plus petite que la distance actuelle de ce voisin, alors la distance actuelle est remplacée par cette nouvelle distance (cf. ligne 148 à 156).

d. Marquage du sommet visité : Une fois que tous les voisins du sommet sélectionné ont été mis à jour, le sommet sélectionné est marqué comme « visité » (cf. ligne 145 à 146).

e. Répétition : Les étapes b. à e. sont répétées jusqu'à ce que tous les sommets du graphe soient visités ou que le sommet d'arrivée soit atteint.

f. Retour du chemin : Une fois que l'algorithme A* a atteint le sommet d'arrivée, il reconstruit le chemin le plus court en partant du sommet d'arrivée et en remontant les distances mises à jour jusqu'au sommet de départ (cf. ligne 136 à 143).

La complexité va elle aussi dépendre de la qualité de l'heuristique. En effet, dans le pire des cas, si l'heuristique est très mauvaise, tous les nœuds devront être explorés et la complexité de l'algorithme sera en O(a*n) où a est le nombre d'arêtes et n le nombre de sommets. Cependant, si l'heuristique est bonne, sa complexité va être comparable à celle de Dijkstra, c'est-à-dire en O((a+n)log(n). Enfin, dans le meilleur des cas, la complexité de l'algorithme A* peut même aller jusqu'à une complexité linéaire.

Cet algorithme a été bien plus compliqué à mettre en place car il fallait bien comprendre chaque étape, comment les formuler et les coder et surtout, gérer la nouvelle fonction qu'est l'heuristique et l'utiliser de la bonne manière.

2) Exécution et tests des programmes

2.1 Hypothèses de travail

Finalement, nous avons trois algorithmes de recherche de plus court chemin à tester. Le premier, dijkstra_mat(), le deuxième dijkstra_dic() et le troisième Astar() (cf. Annexe 3).

Pour rappel, les principales caractéristiques de chaque algorithme sont rappelées dans le tableau ci-dessous.

Algorithme	Complexité	Type de stockage	Heuristique	Temps d'exécution
dijkstra_mat()	$O(n^2)$	Matrice	Non	-
dijkstra_dic()	O(n.log(n))	Dictionnaire	Non	+
Astar()	O(n.log(n))	Matrice	Oui	++

Nos hypothèses de travail sont les suivantes :

a) Si l'algorithme est en complexité O(n.log(n)), alors le temps d'exécution sera plus rapide
b) Si la manière de stocker les données est une matrice, alors de temps d'exécution de l'algorithme sera plus long. L'utilisation d'un dictionnaire, en

revanche, rendrait le temps d'exécution plus rapide puisque les informations seraient plus facilement obtenables.

Nota : nous pensons que notre graphe sera suffisamment grand pour nous permettre de comparer les trois algorithmes que nous avons codés.

2.2 Programme de tests

Modalités

L'ensemble des tests ont été réalisés sur les trois algorithmes de recherche de plus court chemin.

A chaque essai, nous avons utilisé la bibliothèque time de Python et fait plusieurs fois les mêmes mesures dans le but de pouvoir faire une moyenne des temps d'exécution des algorithmes et de comparer les résultats obtenus dans chacun des cas.

Les moyennes ont été réalisé à l'aide d'une boucle itérative que nous avons fait tourner n de fois. Dans cette boucle, nous avions, une première variable tac à laquelle on assignait time.time() qui est le temps en seconde écoulé depuis l'epoch, c'est-à-dire le 1er janvier 1970 à 00 :00 :00.

Ensuite, nous avons exécuter l'algorithme avec une certaine demande (cf. paragraphes suivant). Après cela, il nous a suffi d'ajouter une deuxième variable durée qui calcule la différence entre un nouveau time.time() et tac, ce qui nous permet d'obtenir le temps d'exécution de la recherche d'un plus court chemin (cf. Test.py). Enfin, en faisant tourner la boucle n fois, il ne reste plus qu'à diviser la durée par n pour obtenir une moyenne temporelle.

Programme de test

Premièrement, nous réaliserons un premier test de fonctionnement sur un graphe de petite taille afin de nous assurer que chacun donne le même résultat, et bien sûr de vérifier que ce résultat est cohérent avec le graphe que nous avons entré.

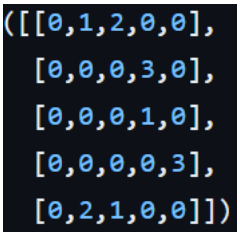
Ensuite, afin de vérifier si les algorithmes fonctionnent toujours avec notre matrice nous avons commencé avec un petit itinéraire sur notre graphe : « Cuire – Hénon ».

Puis, nous avons testé une arrivée sur un croisement : « Hôtel de Ville Louis Pradel ».

Une fois cette vérification faite, nous avons essayé de faire passer notre algorithme par deux intersections pour vérifier que celui-ci renvoie bien, à chaque fois qu'il y a un changement de métro, quelle correspondance prendre.

Enfin, nous avons réalisé un dernier test afin de comparer nos algorithmes de recherche de plus court chemin sur une grande distance : « Gare de Vaise – Vaulx-en-Velin La Soie », afin de mettre en avant le fait que dijkstra_mat() soit moins efficace que les deux autres sur de grand jeu de données.

2.3 Résultats des tests



Le premier test, réalisé sur la matrice d'adjacence ci-contre, a été concluant pour les deux Dijkstra. En effet, en leur demandant le chemin le plus court entre 0 et 4, les deux algorithmes retournent le même chemin : [0,2,3,4,] et la même distance parcourue : 6, ce qui est rassurant. De plus, en se basant sur le graphe que nous leur avions donné, le chemin qui nous est renvoyé est le bon. Cependant, étant donné que ce n'était qu'un petit graphe mathématique sans réelle signification, nous n'avions pas d'heuristique approprié afin de permettre un vrai test de l'algorithme A*. Au niveau de la vitesse d'exécution, au vu de la taille du graphe, la moyenne temporelle obtenue est dans les deux cas égale à zéro seconde.

Le test pour vérifier que les programmes fonctionnent sur notre graphe a été concluant. Aucun des trois algorithmes n'a posé problème pour cette petite étape de vérification et chacun nous a renvoyé le chemin « Cuire Hénon » et une distance de 365 (unité du système ESPG), comme cela était prévu par la lecture du graphe. Encore une fois, la durée d'exécution était nulle.

Ensuite, l'arrivée sur un croisement n'a elle aussi pas engendré d'erreur. Les trois algorithmes sont arrivés au bout et nous ont renvoyé le bon chemin ainsi que la même distance de parcours. Le chemin étant court et sans croisement, la durée d'exécution a une fois de plus été de zéro seconde.

En faisant tourner notre programme avec chacun des algorithmes de recherche de plus court chemin, les changements de lignes n'ont jamais posé problème. A chaque fois qu'un changement doit être réalisé par l'utilisateur, le programme lui renvoie bien l'information. Concernant, l'efficacité des algorithmes, il y a du changement. En effet, les durées d'exécution ne sont plus nulles. Cependant, après avoir fait tourner une boucle 10000 fois sur le chemin « Cuire – Perrache », nous avons obtenu une durée de 3,4 secondes pour dijkstra_mat(), 0,22 seconde pour disjktra_dic() et de 2,0 secondes pour Astar(). Soit, respectivement, un temps d'exécution de 0,34 ms, de 0,022 ms et de 0,20 ms par itération, c'est-à-dire pour une recherche de chemin.

Nous avons fini notre série de tests sur le chemin le plus long de notre graphe, comme prévu. Cette fois-ci, en faisant tourner une boucle 10000 fois, nous obtenons une durée de 25 secondes pour dijkstra_mat(), 0,62 seconde pour dijkstra_dic() et de 10,5 secondes pour Astar(). Ce qui nous donne, respectivement, 2,5 ms, 0,062 ms et 1,05 ms par recherche pour nos algorithmes.

3) Analyse des résultats, interprétation

Tout d'abord, nous pouvons remarquer que l'ensemble des chemins trouvés par nos trois algorithmes sont, à chaque test, identiques. Le A*, qui est censé ne pas forcément trouver le résultat exact mais plutôt un bon résultat mais plus rapidement, a dans notre cas toujours trouvé les chemins exacts.

Ensuite, des résultats obtenus ressort un constat flagrant, dijkstra_dic() est bien plus rapide que les deux autres lorsqu'il s'agit d'un chemin de taille normale. Cela a donc bien été mis en avant lorsque nous avons tester nos trois algorithmes sur le chemin le plus long présent dans notre graphe. En effet, le Astar() met 17 fois plus de temps tandis que le dijsktra_mat() lui met 40 fois plus.

Lorsque l'on traite, en revanche, des chemins de petites tailles, les résultats sont très similaires pour les trois algorithmes.

De ces résultats, nous pouvons en déduire que l'algorithme A* est environ deux fois plus efficace que l'algorithme de Dijkstra lorsque la manière de stockage des données est identique, ce qui était prévu. Nous pouvons donc en déduire que l'heuristique utilisée était adapté. Nous prévoyions également que le fait de stocker les données dans un dictionnaire permettrait au Dijkstra d'être plus efficace et cela s'est avéré bien réel.

En somme, ces tests nous ont permis de valider que le stockage des données est aussi voire plus importante que l'utilisation d'un algorithme réputé plus efficace.

Notre graphe étant de taille moyenne au vu du nombre de stations de métro dans Lyon, ces différences de temps d'exécution ne posent pas de problèmes majeurs pour l'utilisateur : le temps de calcul reste de l'ordre de quelques centièmes de seconde. Il est néanmoins important de noter que si le graphe se complexifie, grâce à l'ajout des lignes de bus et de tramways par exemple, le programme dijkstra_mat() peut rapidement devenir obsolète au regard de sa complexité quadratique qui multiplie par 4 le temps d'exécution lorsque le nombre de station double. Compte tenu de son heuristique efficace, l'algorithme Astar() supporterait mieux la complexification de notre graphe. Mais il semble évident que dijkstra_dic() ne rencontrera aucune difficulté à gérer une augmentation de la complexité et de la taille de notre graphe.

IV) Interface utilisateur

1) Page d'accueil

Afin de mettre en place une interface intuitive et esthétique nous avons choisi d'utiliser la bibliothèque kivy et les widgets issus de kivymd qui permettent de développer des applications mobiles codées avec le langage de programmation python. Cette bibliothèque fonctionne de la manière suivante : l'application est construite à partir d'une classe au sein de laquelle seront écrites toutes les fonctions nécessaires à son fonctionnement. Pour l'interface visuelle, des fichiers d'extension .kv permettront de définir le design de l'application et de ses différents composants.

Nous commençons par définir quelques variables globales appartenant à la classe de l'application :

- zone_texte_destination : contient le nom la station de destination choisie par l'utilisateur ;
- zone_texte_depart : contient le nom de la station de départ choisie par l'utilisateur ;
- dico_changements : représente un dictionnaire qui possède comme clés les noms des stations où il y a des changements et comme valeurs la ligne d'où l'on vient et la ligne où l'on va ;
- distance : contient la distance en kilomètre parcourue au totale ;
- chemin : liste qui contient le nom des stations visitées lors du parcours du graphe.

Ces variables seront accessibles à tout moment mais ne contiennent pas encore leurs données respectives. Nous créons ensuite une première fonction build() qui se lancera au moment de la construction de l'application, c'est-à-dire avant toutes les autres. Cette fonction définie un widget Screenmanager() qui possède deux widgets enfants créés à partir des fichiers page_principale.kv et page_resultat.kv. Ce widget nous permettra de choisir quel écran est affiché. Ensuite, il est nécessaire de créer deux menus déroulants afin que l'utilisateur puisse sélectionner sa station de départ ainsi que celle d'arrivée. Pour faire cela des widgets MDDropDownMenu() peuvent être utilisés à partir de listes comportant les stations sélectionnables. En utilisant des menus déroulants au lieu de simples zones de textes, nous réduisons les erreurs telles que les fautes de syntaxes et contrôlons ainsi les stations que l'utilisateur peut choisir. La fonction build() est maintenant entièrement définie (cf. fichier Main.py ligne 117). Les deux fonctions suivantes sont en lien avec les menus déroulants : elles permettent de mettre à jour dynamiquement le nom de la station choisie par l'utilisateur, que ce soit pour la station de départ ou pour la station de destination. La fonction inverser_stations() permet en un clic sur l'icône des deux flèches de sens opposés d'inverser la station choisie comme départ avec celle choisie comme destination. Enfin, en dessous des deux zones de texte se situe un bouton bleu qui, lorsqu'il est appuyé, appelle la fonction rechercher(). Celle-ci vérifie d'abord si l'utilisateur a bien sélectionné les stations demandées avec la formulation suivante :

```
try :
    [code à exécuter qui peut potentiellement renvoyer une erreur]
except [Erreur prévue] :
    [code à exécuter si le code précédent renvoie l'erreur prévue]
else :
    [code à exécuter si le code n'émet pas l'erreur prévue]
finally :
    [code à exécuter quoi qu'il se passe]
```

Cette formulation est utilisée car même si les variables zone_texte_depart et zone_texte_destination ont été définies au début de la classe, tant que l'utilisateur n'a pas sélectionner les stations demandées, ces deux variables ne possèdent aucun attribut : lorsqu'elles seront appelées, elles renverront l'erreur AttributeError. Ainsi nous pouvons faire apparaitre une boite de message personnalisée si le code rencontre cette erreur. Une fois cette erreur outrepassée, la fonction vérifie si les deux stations spécifiées sont différentes. Si ce n'est pas le cas, elle affiche la même boîte d'erreur définie précédemment mais avec cette fois ci un message différent (cf. Annexe 4). Sinon elle appelle la fonction recherche_itineraire() qui fait elle appel à la fonction de recherche voulue : ici l'algorithme de Dijkstra. Enfin elle affiche la page page_resultat.kv et appelle la fonction afficher_resultats().

2) Page des résultats

Grâce à la fonction recherche_itineraire(), les variables chemin, distance et dico_changements ont pu être définies. Il faut, à partir d'elles, réussir à créer un affichage intuitif et compréhensible. Pour faire cela, nous avons réalisé des illustrations qui représente le chemin entre deux stations. Ces illustrations sont de la couleur de chaque ligne de métro et de funiculaire, il faut donc veiller à afficher l'illustration adaptée à chaque trajet entre deux stations. De plus, en fonction du nombre de changement, il faut adapter la taille des illustrations pour que tous les éléments puissent être affiché à l'écran. A l'aide de boucles conditionnelles, nous avons donc pu établir un affichage dynamique qui s'adapte en fonction du nombre de changements. L'affichage est ainsi terminé (cf. Annexe 4).

V) Conclusion

La mise en place d'algorithme permettant de trouver le plus court chemin entre deux points d'un graphe est très rependue et très utilisée dans de nombreux domaines, comme notamment dans l'industrie du jeu vidéo, dans le domaine de la chimie et surtout dans la recherche d'itinéraire

Nous avons pu réaliser notre propre application nous permettant de guider l'utilisateur sur les choix à faire et sur les métros à prendre en fonction d'où il souhaite aller dans Lyon. Intégrer les tramways puis les bus ne nous a pas été possible par manque de temps et en raison des méthodes utilisées. En effet, la méthode employée pour trier les stations dans l'ordre de parcours du métro n'aurait pas pu être utilisée pour les tramways ainsi que les bus puisque les arrêtes sont beaucoup plus proches et que les bus et tramways réalisent bien plus de virages que les métros.

Le travail réalisé nous permet tout de même de répondre à notre problématique sur l'utilité de la manière dont sont stockées les données et sur le choix de l'algorithme de recherche de plus court chemin à utiliser.

Le stockage se trouve être plus important que l'algorithme utilisé. En effet, un meilleur algorithme va lui-même faire moins de calcul mais si de grandes quantités

de calculs doivent être réalisées pour que celui-ci puisse exploiter les données, cela revient à utiliser un mauvais algorithme qui réalise beaucoup de calculs. De plus, le fait que notre algorithme A* soit assez peu efficace peut provenir du fait que de nombreuses fonctions et variables sont importées depuis d'autres programmes, et utilisées directement à l'intérieur de la fonction, comme par exemple la fonction voisin() (cf. Annexe 2), ce qui peut entrainer une perte de performance.

Afin de mieux traiter notre sujet, nous aurions aimé réussir à implanter les tramways et les bus dans notre graphe pour que notre application soit réellement utile pour un citoyen lyonnais et que l'efficacité des algorithmes que nous avons testés soit vraiment mise à l'épreuve.

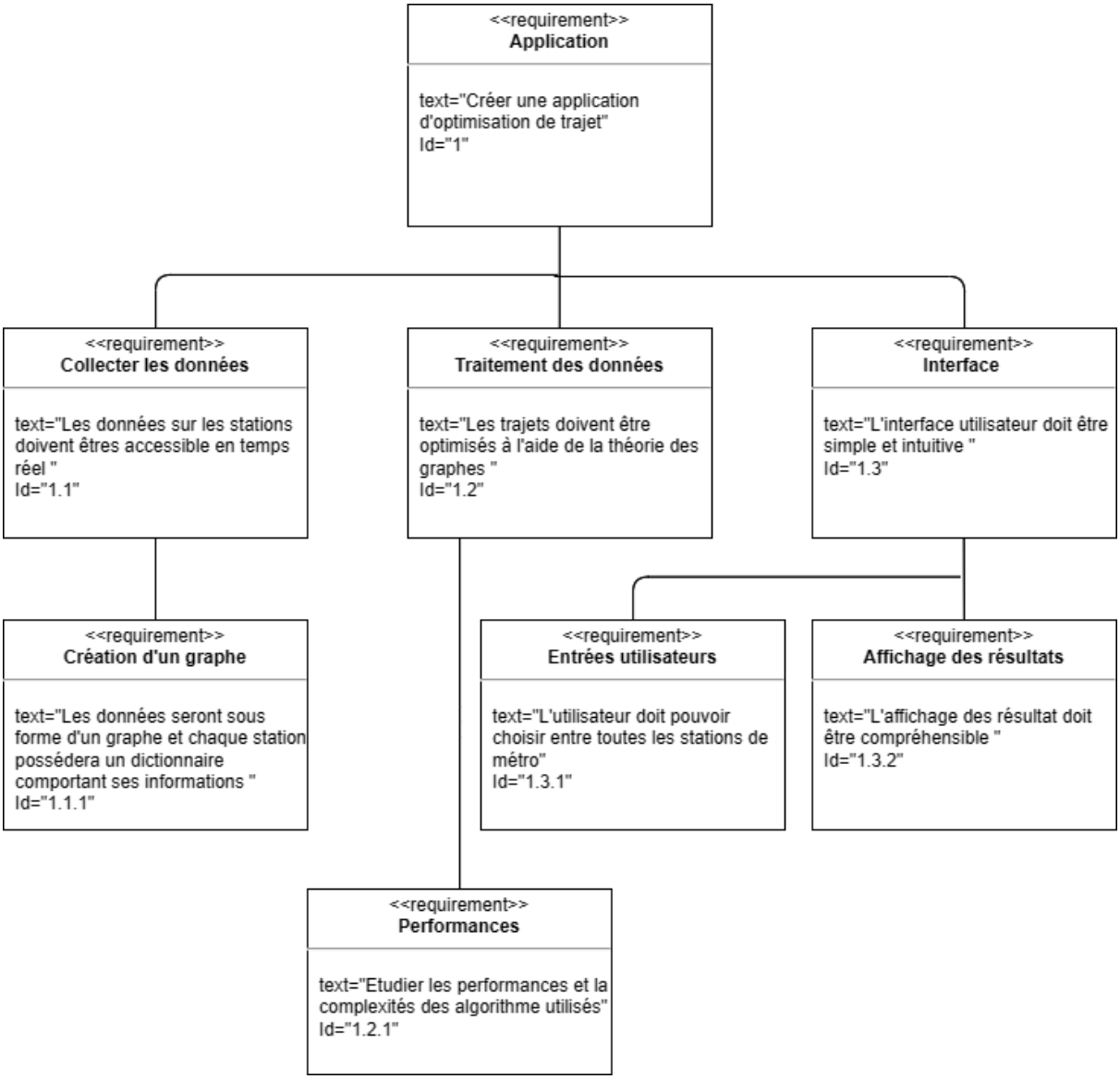
Finalement, la recherche de plus court chemin en théorie des graphes reste un sujet compliqué bien que connu et pas totalement maîtrisé. A ce jour, de nombreux problèmes n'ont toujours pas de solution. Cependant, l'arrivée des intelligences artificielles devraient dans un avenir proche permettre des grandes améliorations dans ce domaine.

ANNEXES

Références bibliographiques

[1] Wikipédia : Problème de plus court chemin : https://fr.wikipedia.org/wiki/Probl%C3%A8me_de_plus_court_chemin
[2] EricSigward : Introduction à la théorie des graphes : https://www.imo.universite-parissaclay.fr/~ruette/mathsdiscretes/polygraph-Sigward.pdf
[3] Michel Rigo : Théorie des graphes : http://www.discmath.ulg.ac.be/cours/main_graphes.pdf
[4] Olivier Laval, Sophie Toulouse, Anass Nagih : Rapport de recherche sur le problème du plus court chemin contraint : https://hal.science/hal-00153962/document
[5] Axel Parmentier : Quelques Algorithmes pour des problèmes de plus court chemin et d'opérations aériennes : https://pastel.archives-ouvertes.fr/tel-01526771/document
[6] J.-F. Scheid : Chapitre 8 : Introduction aux méthodes heuristiques : http://scheid.perso.math.cnrs.fr/Enseignement/GRO/heuristiques.pdf
[7] Wikipédia : Problème du voyageur de commerce :https://fr.wikipedia.org/wiki/Probl%C3%A8me_du_voyageur_de_commerce
[8] TadunfockTeti Bernard, Fotso Laure Pauline : Heuristiques du problème du voyageur de commerce : https://www.cari-info.org/actes2006/144.pdf
[9] Edsger W. Dijkstra, Laurent Beauguitte, Marion Maisonobe : E.W. Dijkstra, 1959, A Note on Two Problems in Connexion with Graphs. Numerische Mathematik 1, p. 269-271 Version bilingue et commentée :https://hal.science/hal-03171590/document
[10] Peter Hart, Nils Nilsson et Bertram Raphael :A Formal Basis for the Heuristic Determination of Minimum Cost Paths : http://ai.stanford.edu/users/nilsson/OnlinePubs-Nils/PublishedPapers/astar.pdf

Annexe 1 : Diagramme d'exigences



Annexe 2 : Fichier Vérification.py

```
""" ********************
    **** Functions ****
    ******************** """

##### Connexité #####

defparcours_profondeur(graphe, accessible, debut):
    accessible[debut] =True
    forvoisinsinrange(len(graphe)):
        ifgraphe[debut][voisins] andnotaccessible[voisins]:
            parcours_profondeur(graphe, accessible, voisins)

defconnexite(graphe):
    visite= [False] *len(graphe)
    parcours_profondeur(graphe, visite, 0)
    forsommetinvisite:
        ifnotsommet:
            returnFalse
    returnTrue

##### Cycle Hamoltonien #####

defcondition_Dirac(matrice): # Verifie la condition de Dirac qui dit qu'un cylce
hamiltonien peut exister seuleument si deg(sommet) >= n/2
    matrice_ligne_sans_zeros= [[elforelinligneifel!=0] forligneinmatrice]
    foriinrange(len(matrice)):
        iflen(matrice_ligne_sans_zeros[i]) < (len(matrice)/2): # la matrice est
symetrique donc len(matrice) correspond au nombre de sommets
            returnFalse
    returnTrue

##### Densité #####

defdensite(graph):
    n=len(graph)
    somme_aretes=0
    foriinrange(n):
        for j inrange(n):
            ifgraph[i][j]!=0:
                somme_aretes+=1
    returnsomme_aretes/(n*(n-1)/2)

##### Centralité #####

defvoisins(matrice,sommet):
    liste_voisins=[iforiinrange(len(matrice)) ifmatrice[sommet][i]!=0]
    returnliste_voisins

defcentralite(graph):
    sommets_principaux={}
    n=len(graph)
    foriinrange(n):
        iflen(voisins(graph,i))>2:
            sommets_principaux[i]=len(voisins(graph,i))
    returnsommets_principaux
```

Annexe 3 : Fichier Algo_PCC.py

```
""" ********************
    **** Imports ****
    ******************** """

importheapq
fromRecuperation_donneesimportrecup
fromRecuperation_donneesimportdistance
fromMatrice_adjacenceimportcreation_matrice

""" ********************
    **** Functions ****
    ******************** """

defvoisins(matrice, sommet):
    liste_voisins=[iforiinrange(len(matrice)) ifmatrice[sommet][i]!=0]
    returnliste_voisins

##########

defdijkstra_mat(mat, start, end):
    asserttype(start) ==int , "L'indice de la station de départ doit être un
entier"
    asserttype(end) ==int , "L'indice de la station d'arrivé doit être un entier"
    n =len(mat)
    distances = [float('inf') foriinrange(n)]
    sommet_pre= [NoneforiinrangeIn(n)]
    visite= [FalseforiinrangeIn(n)]
    distances[start] =0
    file = [start]
    voisins_end=voisins(mat,end)

    whilelen(voisins_end)!=0:
        sommet_act=file[0]
        visite[sommet_act] =True
        delfile[0]
        foriinrange(n):
            ifmat[sommet_act][i] !=0:
                if distances[i] > distances[sommet_act] +mat[sommet_act][i]:
                    distances[i] = distances[sommet_act] +mat[sommet_act][i]
                    sommet_pre[i] =sommet_act
                ifinotin file:
                    file.append(i)
                ifsommet_actinvoisins_end:
                    voisins_end.remove(sommet_act)

    chemin = [end]
    sommet_act=end

    whilesommet_act!=start:
        chemin.append(sommet_pre[sommet_act])
        sommet_act=sommet_pre[sommet_act]

    chemin.reverse()

    return chemin, distances[end]
```

```
##########

defmat_to_dico(matrice):
    dico= {}
    foriinrange(len(matrice)):
        dico[i] = {}
        for j inrange(len(matrice[i])):
            ifmatrice[i][j] !=0:
                dico[i][j] =matrice[i][j]
    return dico

##########

defdijkstra_dic(dico, depart, arrivee):
    asserttype(depart) ==int , "L'indice de la station de départ doit être un
entier"
    asserttype(arrivee) ==int , "L'indice de la station d'arrivé doit être un
entier"
    distances = {sommet :float('inf') forsommetindico}
    distances[depart] =0
    heap = [(0, depart)]
    sommet_pre= {}

    while heap:
        (distance_act, sommet_act) =heapq.heappop(heap)

        ifsommet_act==arrivee:
            chemin= []

            whilesommet_actinsommet_pre:
                chemin.insert(0, sommet_act)
                sommet_act=sommet_pre[sommet_act]

            chemin.insert(0, depart)
            return (chemin, distance_act)

        forvoisins, distance indico[sommet_act].items():
            distance_temp= distances[sommet_act] + distance

            ifdistance_temp< distances[voisins]:
                distances[voisins] =distance_temp
                sommet_pre[voisins] =sommet_act
                heapq.heappush(heap, (distance_temp, voisins))

    returnNone

##########

defAstar(matrice, debut, fin):
    openlist= [debut]
    closedlist= []
    parents = {}
    poids= {debut: 0}
    distances = {debut: 0}

    whilelen(openlist) !=0:

        cout=float("inf")
        fornoeudinopenlist:
```

```
            if poids[noeud]
+distance(stations_infos[liste_stations_matrice[noeud]],
stations_infos[liste_stations_matrice[fin]]) < cout:
                cout= poids[noeud]
+distance(stations_infos[liste_stations_matrice[noeud]],
stations_infos[liste_stations_matrice[fin]])
                noeud_act=noeud

        ifnoeud_act==fin:
            chemin= [fin]
            dernier=fin
            while dernier !=debut:
                chemin.append(parents[dernier])
                dernier= parents[dernier]
            chemin.reverse()
            return chemin, distances[fin]

        closedlist.append(noeud_act)
        openlist.remove(noeud_act)

        forvoiinvoisins(matrice, noeud_act):
            ifvoinotinclosedlist:
                new_poids=poids[noeud_act] +matrice[noeud_act][voi]
                ifvoinotinopenlistornew_poids<poids[voi]:
                    parents[voi] =noeud_act
                    poids[voi] =new_poids
                    distances[voi] = distances[noeud_act]
+matrice[noeud_act][voi]
                    ifvoinotinopenlist:
                        openlist.append(voi)

    return"Impossible d'atteindre l'arrivée", "distance inconnue"

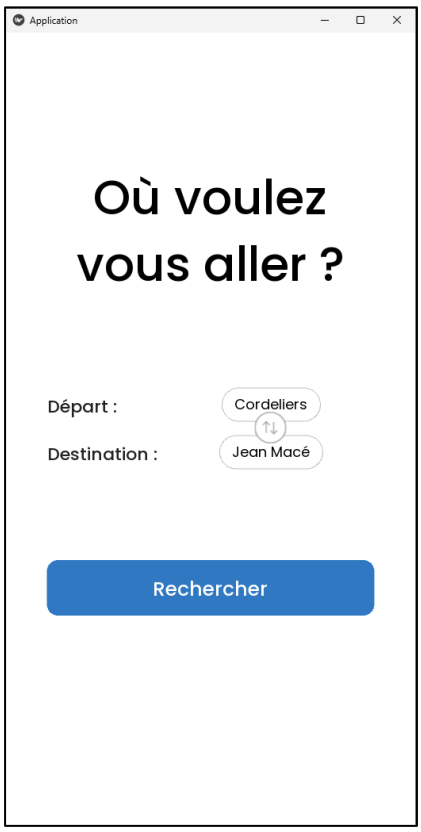
""" **************
    **** Main ****
    ************** """

lien_lignes="https://download.data.grandlyon.com/wfs/rdata?SERVICE=WFS&VERSION=2.0
.0&request=GetFeature&typename=tcl_sytral.tcllignemf_2_0_0&outputFormat=applicatio
n/json; subtype=geojson&SRSNAME=EPSG:3946&startIndex=0&count=100"
lien_arrets="https://download.data.grandlyon.com/wfs/rdata?SERVICE=WFS&VERSION=2.0
.0&request=GetFeature&typename=tcl_sytral.tclarret&outputFormat=application/json;
subtype=geojson&SRSNAME=EPSG:3946&startIndex=0&count=4657"

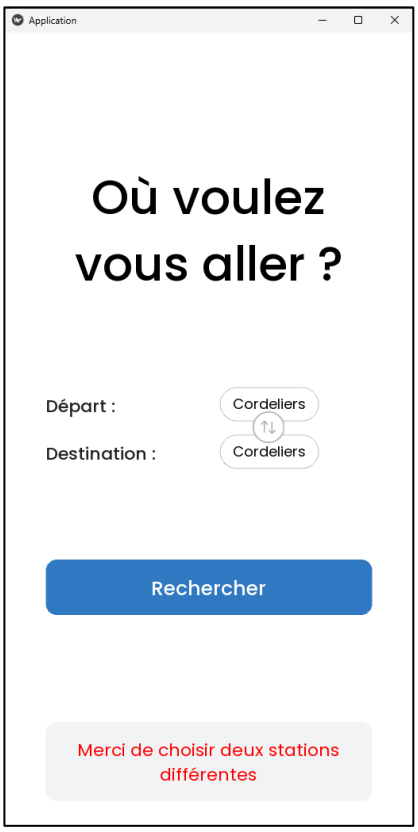
stations_infos,lignes_infos,codes_lignes=recup(lien_lignes, lien_arrets)

matrice,liste_stations_matrice=creation_matrice(lignes_infos, stations_infos)
```

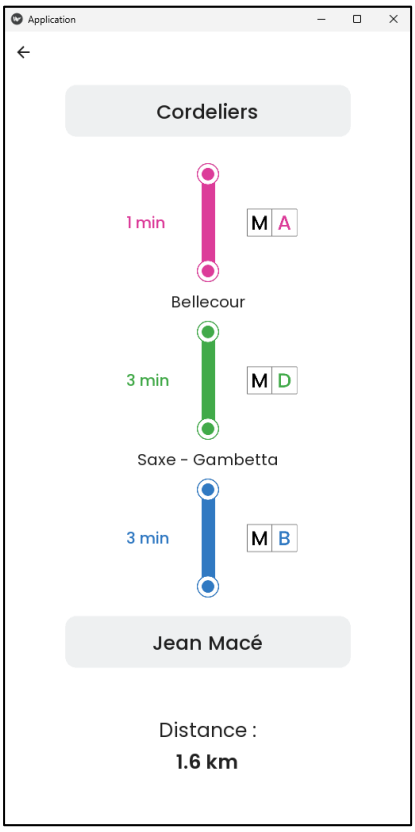
Annexe 4 : Interface utilisateur



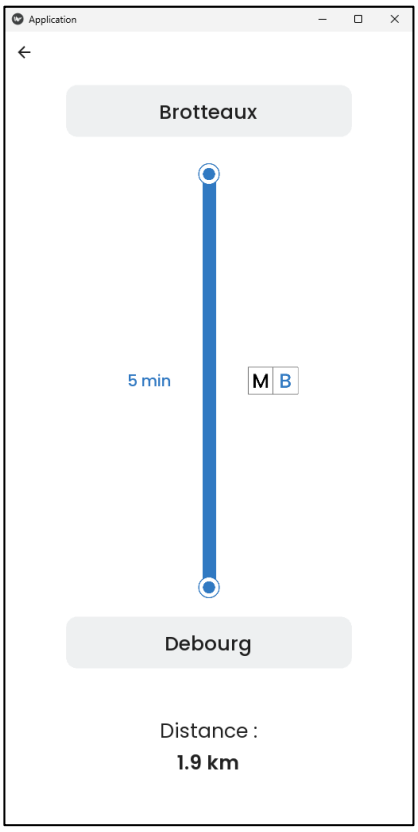
Page d'accueil



Message d'erreur



Page des résultats



Autre exemple

Résumé

L'objectif de ce projet TIPE était de développer une application d'optimisation de trajet entre deux stations de métro en utilisant la théorie des graphes. Pour ce faire, nous avons étudié les concepts fondamentaux de la théorie des graphes tels que les graphes pondérés, les algorithmes de recherche de chemin et les algorithmes d'optimisation.

Nous avons ensuite collecté des données sur le site **data.grandlyon.com** des différentes stations de métro et des connexions entre elles pour créer un graphe représentant le réseau de métro. En utilisant cet outil, nous avons développé un algorithme permettant de trouver le chemin le plus court entre deux stations données, en tenant compte des contraintes telles que les changements de ligne et les correspondances.

Enfin, nous avons implémenté cette solution sous la forme d'une application intuitive pour les utilisateurs, permettant de trouver le trajet optimal entre deux stations de métro en un minimum de temps et de kilomètres.

Les résultats de cette recherche montrent que l'utilisation de la théorie des graphes pour l'optimisation des trajets en métro peut considérablement améliorer l'expérience de voyage des utilisateurs, ce qui s'inscrit dans le thème de la mobilité en ville.

Abstract

The objective of this project was to develop a route optimization application between two metro stations using graph theory. To do this, we studied the fundamental concepts of graph theory such as weighted graphs, pathfinding algorithms, and optimization algorithms.

Next, we collected data on the **data.grandlyon.com** website on the various metro stations and connections between them to create a graph representing the metro network. Using this tool, we developed an algorithm to find the shortest path between two given stations, taking into account constraints such as line changes and connections.

Finally, we implemented this solution in the form of an intuitive application for users, allowing them to find the optimal route between two metro stations in a minimum amount of time and distance.

The results of this research show that the use of graph theory for metro route optimization can significantly improve the travel experience of users, which is in line with the theme of mobility in the city.